Blockchain aplikacije, Predavanje 01

Od raspodijeljenih do decentraliziranih sustava

Fakultet informatike, Sveučilište Jurja Dobrile u Puli

izv. prof. dr. sc. Nikola Tanković

Nacrt za nastavnikov pregled. Predavanje traje 120 minuta s raspravom, zadacima i ponavljanjem. Engleski prijevod slijedi nakon pregleda hrvatske verzije.

1 Vodič kroz predavanje

Polazimo od pitanja tko kontrolira evidenciju zajedničkog digitalnog budžeta. Konkurentne isplate 80 i 60 iz salda 100 kasnije služe za analizu promjena stanja. Od jednog koordinatora prelazimo na replike, zatim na kvarove i neovisne operatore. Predznanje: zapis podataka, uvjet i poziv funkcije. Crteži i pseudokod prikazuju nastavne modele; ne predstavljaju gotov distribuirani protokol.

2 Što je ledger?

Ledger, na hrvatskom glavna knjiga, uređen je zapis svih knjiženja po računima. Iz njega se u svakom trenutku može izračunati saldo svakog računa, vidjeti tko je kada što promijenio i usporediti vlastite zapise s tuđima. U dvojnom knjigovodstvu (*double-entry bookkeeping*) svaka transakcija upisuje se dvaput: jednom na teret jednog računa i jednom u korist drugog, istim iznosom. Dictionary.com / Collins: double entry. Zato zbroj salda ostaje isti, a pogrešno knjiženje pokazuje se kao neslaganje.

Knjiženje	Račun	Terećenje	Odobrenje	Saldo
početak	Ana / Boris			0 / 0
gotovinska uplata	Ana		100	100
gotovinska uplata	Boris		20	20

Knjiženje	Račun	Terećenje	Odobrenje	Saldo
K1: Ana plaća Borisu 30	Ana	30		70
	Boris		30	50
K2: Boris plaća Ani 10	Boris	10		40
	Ana		10	80

Koristimo bankovne izraze terećenje i odobrenje; knjigovodstvena pravila u punom opsegu nisu cilj kolegija. Isplata tx_1 iz našeg budžeta kasnije će biti upravo takvo knjiženje: terećenje računa F, odobrenje računa X.

Gotovinska uplata razlikuje se od knjiženja K1 i K2: novac dolazi izvana, pa banka drugu stranu knjiženja vodi na svom računu blagajne, koji se na izvodu klijenta ne vidi. K1 i K2 samo premještaju novac između dvaju računa.

Nastavni rad (1–3 min): Pitajte koliko je ukupno novca na oba računa nakon uplata i nakon svakog knjiženja. Uvijek 120.

3 Ledger banke: sva knjiženja na jednom mjestu

Banka ne vodi zaseban zapis za svakog klijenta, nego jednu glavnu knjigu za sve račune, uključujući vlastitu blagajnu. Izvod klijenta samo je pogled na retke jednog računa iz te knjige. Gotovinska uplata ima drugu stranu na računu blagajne banke, jer novac dolazi izvana; K1 i K2 samo premještaju novac između računa klijenata.

Datum	Knjiženje	Račun	Terećenje	Odobrenje
1. 3.	U1: gotovinska uplata	Blagajna banke	100	
		Ana		100
1. 3.	U2: gotovinska uplata	Blagajna banke	20	
		Boris		20
2. 3.	K1: Ana plaća Borisu	Ana	30	
		Boris		30
5. 3.	K2: Boris plaća Ani	Boris	10	
		Ana		10
	Ukupno		160	160

Svako knjiženje ima dva retka s istim iznosom, pa su zbrojevi terećenja i odobrenja uvijek jednaki; knjiženje s jednom stranom odmah se vidi. Ovo je pojednostavljeni nastavni prikaz: ne ulazimo u to da su za banku računi klijenata obveze, a blagajna imovina.

Nastavni rad (3–5 min): Pitajte odakle na Aninu izvodu dolazi +100 i gdje je druga strana tog knjiženja.

4 Kako surađuju banke s odvojenim evidencijama?

Banke ne moraju dati jednoj drugoj administratorsku kontrolu nad svim svojim evidencijama. Vode zasebne zapise, razmjenjuju naloge i koriste dogovorene postupke obračuna obveza i njihova poravnanja. U jednom konkretnom sustavu, tx_2, međubankovne se obveze poravnavaju prijenosom novca između računa kod središnje banke u stvarnom vremenu. ECB: tx_2. Sud nije korak kroz koji prolazi svako plaćanje: pravna zaštita služi rješavanju sporova, dok poravnanje ima vlastita pravila i infrastrukturu. U redovnom radu evidencije se slažu: nalozi između dviju banaka skupljaju se tijekom ciklusa, a poravnava se samo razlika, pa A plati B ili B plati A (netiranje; velika i hitna plaćanja idu pojedinačno i u stvarnom vremenu). Evidencije se ne slažu zato što banke međusobno uspoređuju knjige, nego zato što sve vjeruju središnjoj banci i oslanjaju se na njezin zapis. Ako se pojavi razlika, npr. pogrešan iznos ili dvostruki nalog, rješava se povratom ili opozivom po pravilima platne sheme. Sud dolazi tek kod spora o samom poslu. To je slučaj posrednika od povjerenja iz predavanja 00. Račun u tx_2 mogu imati kreditne institucije, a od 2025. i platne institucije i institucije za e-novac pod istim uvjetima (samo sredstva za poravnanje tog dana, bez čuvanja novca korisnika; ECB, 19. 7. 2024.); građani i obične tvrtke ne mogu. Naknade prema ECB TARGET Services Pricing Guide S_3.0, srpanj 2026.: račun za plaćanja (RTGS) 400 € mjesečno + 0,80 € po nalogu ili 5.000 € mjesečno + 0,60–0,05 € po nalogu; TIPS 800 € mjesečno + 0,002 € po trenutnom plaćanju. Revizija cijena RTGS-a 2026.: provjeriti jesu li navedeni iznosi već nove vrijednosti. Usporedba: pristup posredniku daje licenca uz fiksne troškove i nadzor; javni *blockchain* otvoren je svima, ali se plaća svaki zapis.

Naš zajednički digitalni budžet pojednostavnjuje pitanje zajednički prihvaćenog stanja; nije model cjelokupnog bankovnog poslovanja. Organizacije mogu prihvatiti operatora, ugovor, reviziju i nadležni sud. Ako zapisi postanu nespojivi, treba prikupiti dokaze, uskladiti evidencije ili riješiti spor. Želimo istražiti može li zajedničko provođenje preciznih pravila spriječiti dio takvih neslaganja prije nego što nastanu.

Zato početno pitanje nije kako računati dvije istodobne isplate. Pitanje je tko smije konačno prihvatiti ili promijeniti zajednički zapis i na čiji se autoritet ostali oslanjaju. Najprije ćemo vidjeti što rješava pouzdan koordinator, zatim replike i na kraju neovisni operatori. Isplate 80 i 60 kasnije daju konkretno pravilo za provjeru tih rješenja.

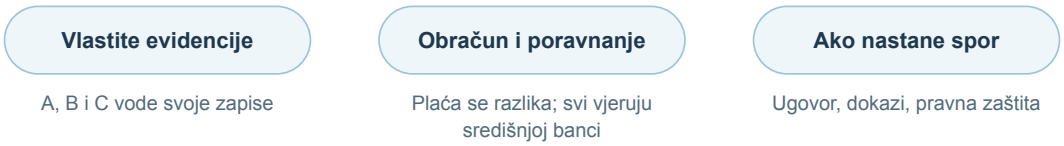


Figure 1: Redovito poravnanje i sudsko rješavanje spora različiti su postupci

Nastavni rad (5–7 min): Pitajte što rješava poravnanje, a što sud. Zatim zatražite primjer troška koji nastaje tek kada se sudionici ne mogu usuglasiti.

5 Pravni put postoji, ali koliko traje i košta?

Europska komisija u izvješću o vladavini prava za Hrvatsku iz 2025., str. 6, navodi prosječno trajanje prvostupanjskih trgovačkih parnica od 1.147 dana u 2024. To je oko 3,1 godine. Brojka opisuje tu kategoriju postupaka i godinu; nije trajanje svake parnice, međubankovnog poravnanja ili posebno sporova o digitalnim zapisima. Ne uključujemo je kao prognozu današnjeg pojedinačnog predmeta. Izvor.

Pravni postupak uključuje pristojbe i moguću naknadu za zastupanje; dodatni dokazni postupci mogu tražiti dodatne izdatke. Iznos ovisi o predmetu i radnjama, pa nema jedne poštene cijene za sve sporove. Europski portal e-pravosuđa. Uz to, organizacija troši vrijeme na dokumentaciju i upravljanje sporom.

Odvojeno računamo autorsku ilustraciju: ako je 100.000 eura zbog spora nedostupno tri godine i pretpostavimo jednostavni trošak financiranja od 5% godišnje, dobivamo 15.000 eura. To nije sudska tarifa, zakonska kamata, opaženi prosjek ni tvrdnja da je novac blokiran u svakoj parnici. Brojevi pokazuju zašto i vrijeme može biti skupo. Spor se može riješiti i dogovorom ili drugim odgovarajućim putem; ne tvrdimo da svako neslaganje završava višegodišnjom parnicom.

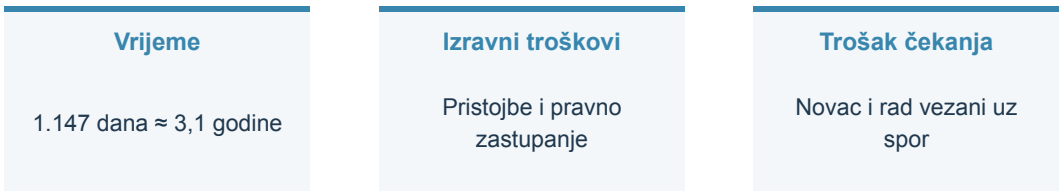


Figure 2: Hrvatska, 2024.: prosjek prvostupanjskih trgovačkih parnica

Nastavni rad (7–9 min): Odvojite na ploči izmjereni podatak 1.147 dana od pretpostavljenog izračuna 15.000 €. Pitajte gdje nastaje poslovni trošak čekanja.

6 Možemo li dio sporova spriječiti unaprijed?

Za naš digitalni budžet predlažemo promjenu dizajna: dopuštene radnje unaprijed preciziramo i sudionici neovisno provjeravaju prijedloge. Trebaju i zajednički postupak prihvaćanja. Sama lokalna provjera ili kopija baze nije dovoljna. Cilj je, pod izričitim pretpostavkama protokola, spriječiti da dva ispravna sudionika nespojive promjene smatraju konačno prihvaćenima.

To je istraživačko pitanje predavanja, a ne unaprijed dokaz da je blockchain jeftiniji. Dio neslaganja o digitalnom stanju mogao bi se spriječiti ili lakše provjeriti, ali sustav plaća komunikaciju, održavanje i složenije upravljanje. Pogrešna pravila, pogreške u programu i pitanja izvan digitalne evidencije ostaju mogući izvori spora. Tehničko prihvaćanje zapisa ne ukida pravne zahtjeve i sudsku zaštitu.

Pouzdana zajednička baza također može automatizirati pravila i smanjiti pogreške. Decentralizirani pristup postaje zanimljiv kada sudionici uz to ne žele jednom operatoru dati isključivu ovlast nad prihvaćenim stanjem. Tijekom predavanja uspoređujemo tu korist s cijenom i preostalim povjerenjem. Konačan izbor mora slijediti potrebe aplikacije.

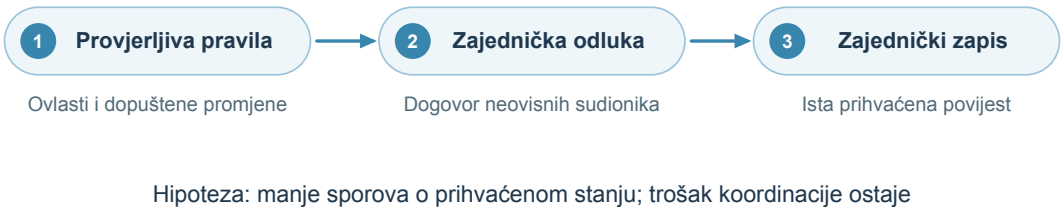


Figure 3: Hipoteza: manje sporova o prihvaćenom stanju; trošak koordinacije ostaje

Nastavni rad (9–11 min): Tražite jedan spor koji provjera salda može spriječiti i jedan koji ostaje moguć čak uz zajednički zapis.

7 Ishodi i predznanje

Kroz cijelo predavanje pratimo isti zapis i iste sudionike. Ne tražimo da zapamtite popis protokola. Cilj je da na crtežu pokažete tko može što učiniti, koju informaciju svaki sudionik ima i koju pretpostavku vaš zaključak zahtijeva. Riječi poput „sigurno”, „dostupno” ili „decentralizirano” nisu dovoljne bez objašnjenja.

Redoslijed rada je namjeran: prvo odredimo poslovno pravilo, zatim ga pokušamo provesti na jednom mjestu, pa dodajemo kopije i ispade. Tek kad organizacije odbiju zajedničku kontrolu, prelazimo na drukčiji model povjerenja. Na kraju treba moći objasniti zašto dodavanje poslužitelja samo po sebi nije riješilo taj problem.

- Razlikovati raspodijeljenost, decentralizaciju i P2P.

- Pratiti promjene stanja kroz više replika.
- Razlikovati kašnjenje, djelomičan ispad i bizantsko ponašanje.
- Navesti pretpostavke povjerenja za predloženo rješenje.
- Obrazložiti kada je zajednički koordinator dovoljan.

Nastavni rad (11–12 min): Najavite 120 minuta uključujući kratke zadatke i raspravu. Provjerite poznaju li studenti JSON zapis i jednostavnu funkciju.

8 Kako prikazujemo stanje i transakcije

Za ostatak kolegija koristimo dva pojma i dva znaka. Stanje (*state*) S_x je cijeli zapis nakon x prihvaćenih transakcija. Transakcija (*transaction*) je jedna dopuštena promjena koja vodi iz stanja S_x u S_{x+1}. Ledger čuva transakcije redom; stanje dobijemo tako da ih primijenimo jednu po jednu, počevši od S_0. Primjer nastavlja izvode s Anom i Borisom: nakon U1, U2, K1 i K2 stanje S_4 daje upravo salda 80 i 40.

Iz toga slijedi važno svojstvo: isti ledger, primijenjen istim redom i po istim pravilima, uvijek daje isto stanje. Zato će kasnije ključno pitanje biti tko određuje redoslijed transakcija. Na crtežima je stanje plava kartica, transakcija narančasta pilula, a oznaka stanja S_x obojeni čip.

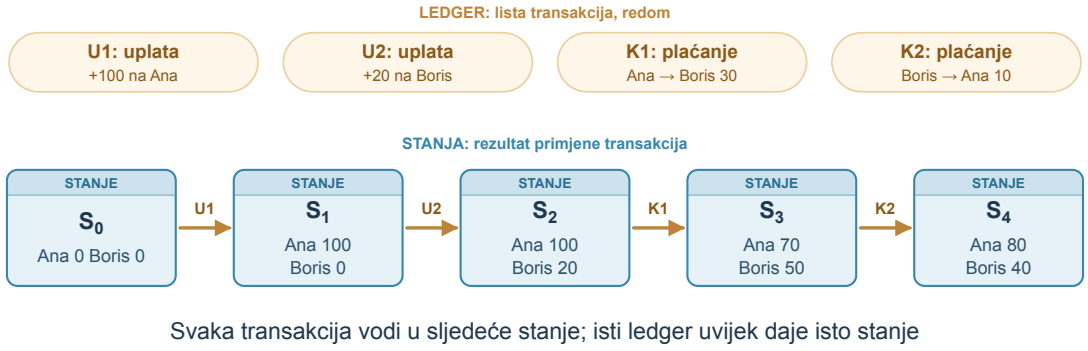


Figure 4: Svaka transakcija vodi u sljedeće stanje; isti ledger uvijek daje isto stanje

Nastavni rad (12–14 min): Pitajte: ako dvije banke imaju isti ledger, mogu li imati različito stanje?

9 Tri organizacije, jedan digitalni budžet

Organizacije A, B i C unaprijed su poznati članovi. Svaki smije predložiti isplatu iz fonda F, ali prijedlog nije odluka. O prihvaćanju odlučuje protokol, kojem se vraćamo kod pravila o saldu. Promatramo jedan trenutak pri stanju S_7, kada fond F ima 100 jedinica, a primatelji X i Y nemaju ništa: B predlaže tx_1, C predlaže tx_2, a A u tom trenutku ne predlaže ništa.

Uloge nisu stalne; u drugom trenutku A može predložiti isplatu, a B mirovati. U primjeru je tx_1 prijenos 80 jedinica iz F u X, a tx_2 prijenos 60 jedinica iz F u Y. X i Y su odredišni računi u istoj digitalnoj evidenciji; oznake A, B i C označavaju sudionike i njihove čvorove, a ne odredišta isplate.

Zahtjev mora doći od ovlaštenog člana, imati pozitivan iznos i dovoljno pokrića u stanju u kojem se prihvaća. Stanja numeriramo: S_x je stanje nakon x prihvaćenih promjena. Indeks x označava korak prihvaćene povijesti, a ne vrijeme na zidnom satu; u kodu ga zovemo verzija. Prvo ćemo svim organizacijama dati zajedničkog koordinatora. Kasnije mijenjamo pretpostavku da mu sve vjeruju.

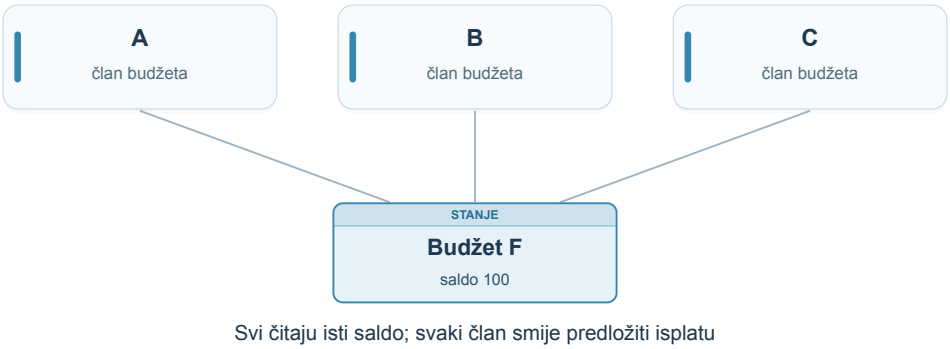


Figure 5: Svi čitaju isti saldo; svaki član smije predložiti isplatu

Nastavni rad (14–17 min): Razdvojite na ploči organizacije A/B/C od računa F/X/Y. Tko predlaže, a tko konačno prihvaća isplatu?

10 Stanje zapisujemo izričito

Početno stanje zapisujemo kao $S_7 = (\mathrm{F}{:}100, \mathrm{X}{:}0, \mathrm{Y}{:}0)$. tx_1 je prijedlog isplate 80 jedinica iz F u X uz očekivano stanje S_7. Sustav provjerava ovlast podnositelja, pozitivan iznos, pokriće i očekivano stanje. Prijedlog ne mijenja stanje samim slanjem.

Prihvaćeni tx_1 daje $S_8 = (\mathrm{F}{:}20, \mathrm{X}{:}80, \mathrm{Y}{:}0)$. Odbijeni zahtjev ne mijenja stanje. Obje strane prijenosa trebaju biti jedna atomarna promjena: nije dovoljno samo smanjiti F ili samo povećati X. Zbroj salda u zatvorenom primjeru ostaje 100. U nastavku dijagrami često pokazuju samo saldo F i oznaku stanja, ali krediti primatelja ostaju dio istog stanja.

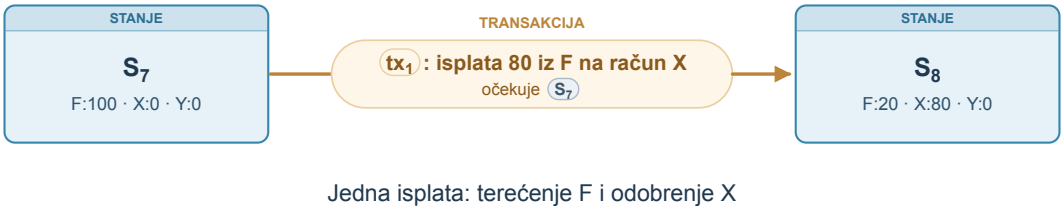


Figure 6: Jedna isplata: terećenje F i odobrenje X

U istom trenutku C predlaže tx_2: isplatu 60 jedinica iz F u Y, također uz očekivano stanje S_7. Prihvaćeni tx_2 dao bi drugo stanje, $S_8' = (\mathrm{F{:}40},\ \mathrm{X{:}0},\ \mathrm{Y{:}60})$. S_8' je alternativno stanje: isti korak od S_7, ali druga povijest.

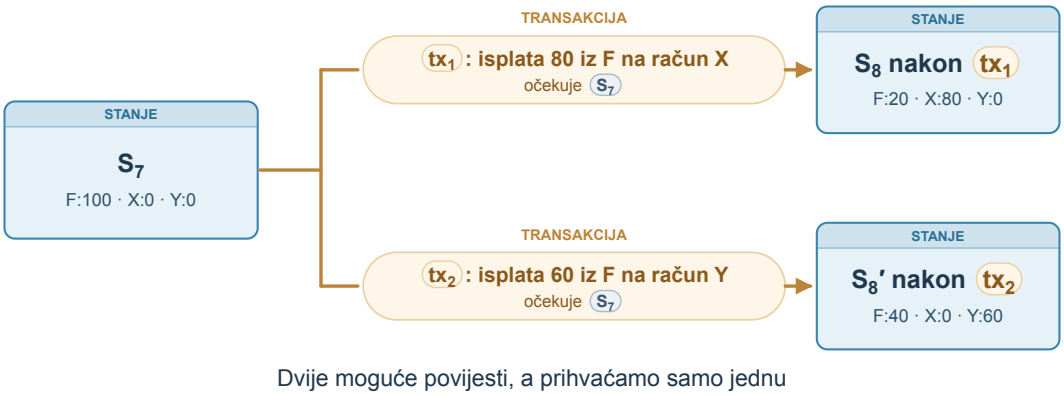


Figure 7: Dvije moguće povijesti, a prihvaćamo samo jednu

Nastavni rad (17–19 min): Neka student izračuna sva tri salda nakon tx_1. Provjerite zbroj i naglasite razliku između zahtjeva i prihvaćene promjene.

11 Pravilo koje mora ostati istinito

Poslovno pravilo ima dvije strane. Prvo, isti novac ne smije se potrošiti dvaput: tx_1 i tx_2 obje polaze od stanja S_7 i svaka zasebno prolazi provjeru pokrića, ali obje zajedno trošile bi istih 100 jedinica. To je dvostruko trošenje (*double spending*). Ovdje ga uvodimo kao problem zajedničkog stanja; u predavanju 04 vidjet ćemo kako ga Bitcoin rješava bez koordinatora. Drugo, prihvaćena isplata ne smije potrošiti više jedinica nego što fond trenutačno ima. U zatvorenom modelu ne stvaramo ni uništavamo jedinice: terećenje jednog računa i odobrenje drugog čuvaju ukupni zbroj. Ovlašten podnositelj također ne smije zaobići pokriće.

Kao dodatni mehanizam naš API koristi očekivano stanje. Iz istog stanja prihvaća najviše jednu promjenu; ostale klijent mora ponovno razmotriti nad novim stanjem. To je namjerno konzervativan mehanizam zaštite od konkurentnih upisa, a ne pravilo da su svake dvije isplate nespojive. Primjerice, 30 i 40 mogu proći redom uz osvježeno očekivano stanje. Naših 80 i 60 ne mogu proći ni u jednom redoslijedu bez novih sredstava.

O tome koje stanje prihvaćamo odlučuje protokol: unaprijed dogovorena pravila koja dijele svi članovi i kojih se svi moraju držati (tko smije predložiti, što se provjerava i kada je isplata prihvaćena). Prihvatiti možemo samo jedno od S_8 i S'_8. Ostaje pitanje tko provodi to pravilo i bira koja isplata prolazi.

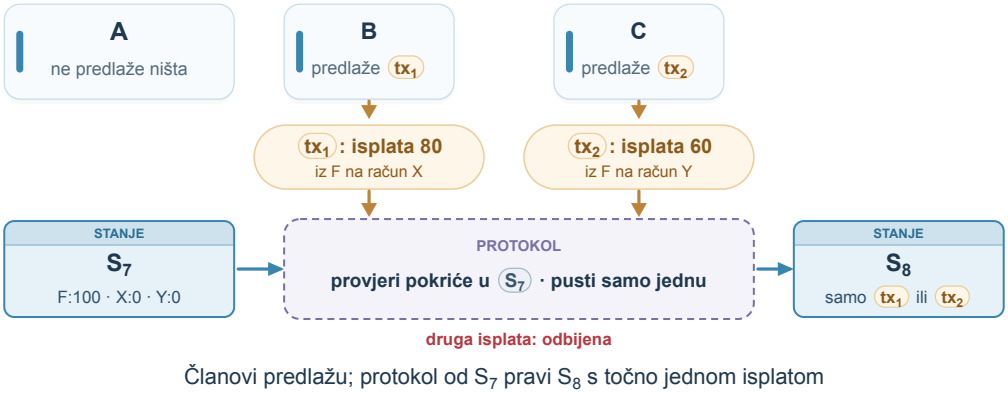


Figure 8: Članovi predlažu transakcije; protokol odlučuje koja mijenja stanje

Nastavni rad (19–22 min): Usporedite parove 80+60 i 30+40. Razlikujte nedostatak pokrića od zastarjelog očekivanog stanja.

12 Prvo rješenje: zajednički koordinator

Prvi korak u izgradnji sustava je jedna zajednička služba s bazom podataka. Služba radi na čvoru kod organizacije A; zato je A u nastavku čvor koji prihvaća upise, a kasnije i član koji odobrava. Član šalje zahtjev, služba provjerava njegovu ovlast, pozitivan iznos, raspoloživi saldo i očekivano stanje. Zatim atomarno mijenja oba računa i bilježi ishod. Ostali čitaju prihvaćeno stanje iz te službe. Zaštita od dvaju istodobnih upisa tada je lokalni problem baze i aplikacije.

Ovo rješenje nije pogrešno zato što je centralizirano. Ako organizacije vjeruju operatoru i prihvaćaju njegove ovlasti, može biti dovoljno. Treba imenovati granicu povjerenja: operator upravlja kodom, pristupom bazi i oporavkom. Klijenti ne odlučuju o konačnom stanju samo zato što na zaslonu imaju njegovu kopiju.



Figure 9: Svi prihvaćaju autoritet istog operatora

Nastavni rad (22–25 min): Povežite s BA-01, str. 40: zadržan centralizirani pristup, promijenjen primjer. Pitajte tko može promijeniti pravila.

13 Dva zahtjeva, jedan redoslijed

Koordinator prvo odabere tx_1, provjeri uvjete i atomarno prenese 80 iz F u X. Preostaje F:20, X:80, Y:0 u S_8. Kada provjeri tx_2, očekivano stanje S_7 više nije aktualna. Čak i kada bi podnositelj obnovio tx_2 nad stanjem S_8, traženih 60 nema pokriće u preostalih 20. Odbijanje je normalan poslovni odgovor, ne kvar mreže.

Da je tx_2 proveden prvi, stanje bi bilo F:40, X:0, Y:60, S_8, a tx_1 bi bio odbijen. Pravilo ne nalaže da baš tx_1 pobijedi. Nalaže dopušten redoslijed, nenegativan saldo i jednake prihvaćene promjene kod sudionika. U ovom primjeru nema dodatnih uplata između zahtjeva. Pritom smo prešutno pretpostavili da koordinator obrađuje zahtjeve jedan po jedan.

Korak	**Radnja**	**Saldo F**
0	Početak	100, S_7
1	tx_1: isplati 80, očekuje S_7	20, S_8
2	tx_2: isplati 60, očekuje S_7	odbijen; 20, S_8

Nastavni rad (25–28 min): Prvo sakrijte zadnji redak i tražite predviđanje. Zatim promijenite redoslijed $\mathrm{tx}_1/\mathrm{tx}_2$ na ploči.

14 Koordinator s dvije dretve: obje provjere prije promjene

Drugi korak je skaliranje koordinatora. Dosad je koordinator obrađivao zahtjeve jedan po jedan, a stvarna služba obrađuje zahtjeve u više dretvi istodobno. tx_1 pročita saldo 100 i stanje S_7, a tx_2 pročita isto. Oba prođu provjeru. Ako zatim bez ponovne provjere provedu

terećenja 80 i 60, saldo F postaje −40, X ima 80, a Y 60. Ukupni zbroj ostaje 100, ali fond je potrošio sredstva koja nema. Ako umjesto terećenja kod prepisuje saldo iz zastarjele kopije, može nastati drukčiji problem: izgubljeni upis i nesklad salda i isplata.

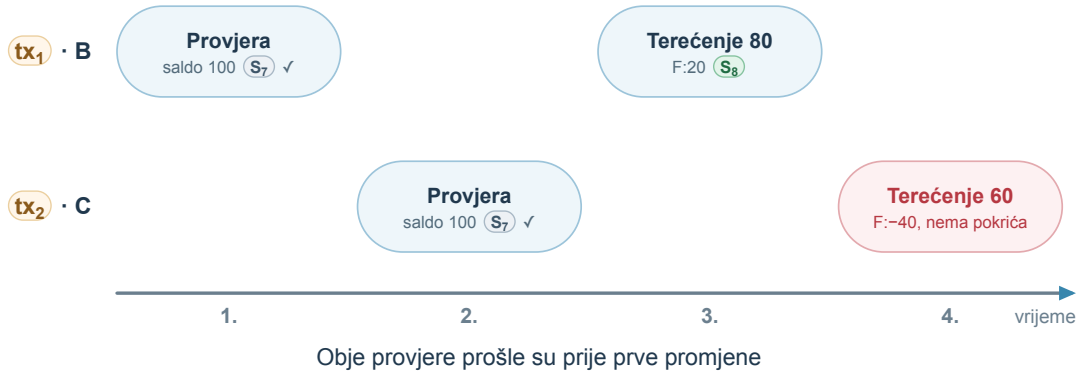


Figure 10: Obje provjere prošle su prije prve promjene

Pseudokod namjerno nema zaštitu koja povezuje provjeru s promjenom. Čak i ako pomoćna funkcija atomarno prenosi iznos između računa, ranija provjera pokrića nije time zaštićena. Pozitivan iznos i ovlast jesu potrebni, ali ne rješavaju konkurentnost. Studenti trebaju pokazati točan raspored radnji koji ruši pravilo. Ni provjera `verzija == ocekivana` ne pomaže: obje dretve pročitaju S_7 prije nego što ijedna poveća verziju, jer su provjera i promjena dva odvojena koraka. Ishod koji ovisi o slučajnom preplitanju dretvi zove se utrka (*race condition*). Ovdje je to problem unutar jednog koordinatora; sljedeći odjeljak pokazuje kako ga baza sprječava.

```
if ovlasten(clan) and iznos > 0:
    if verzija == ocekivana and saldo_F >= iznos:
        tereti_F_i_odobri_primatelja(iznos)
        verzija += 1
        return "prihvaćeno"
```

Što ako oba zahtjeva provjere saldo 100 prije prve promjene?

Nastavni rad (28–31 min): Odglumite: tx_1 provjeri 100, tx_2 provjeri 100, tx_1 tereti 80, tx_2 tereti 60. Izračunajte saldo nakon svakog koraka.

15 Jedna baza: atomarnost i izolacija

Ovdje govorimo o transakciji u jednoj bazi kod pouzdanog koordinatora, a ne o dogovoru neovisnih organizacija. Atomarnost (*atomicity*) osigurava da se terećenje F i odobrenje

primatelja provedu zajedno ili poništе zajedno. Izolacija (*isolation*) određuje dopušteno preplitanje konkurentnih transakcija. Sama atomarnost ne sprječava dva zahtjeva da provjere isti stari saldo.

U našem konkretnom rješenju transakcija počinje naredbom BEGIN. Dohvatimo i zaključamo redak fonda za izmjenu, primjerice SELECT … FOR UPDATE, pa provjerimo ovlast, pozitivan iznos, pokriće i očekivano stanje. Dok držimo lokot, drugi upis u isti fond čeka. Zatim ažuriramo F i račun primatelja te broj stanja i završavamo s COMMIT. Klijentu šaljemo potvrdu tek nakon uspješnog COMMIT-a; ako uvjet ne vrijedi ili nastane pogreška, transakciju poništavamo. Sljedeći zahtjev nakon čekanja provjerava aktualno stanje. Sve isplate moraju slijediti isti postupak.

Pretpostavljamo fiksne ovlasti članova tijekom ovog primjera i jednu bazu koja sadrži sve račune. Ažuriranje primatelja također mora izbjeći izgubljeni upis; za opće prijenose između više računa trebaju dosljedan redoslijed zaključavanja i obrada deadlocka. Primjer je konceptualni slijed, ne kompletna implementacija. Uvjetni upisi ili serijalizabilna izolacija uz obradu ponovnih pokušaja druge su mogućnosti. PostgreSQL 18: zaključavanje redaka.

To štiti poslovno pravilo unutar baze, ali ne uklanja povjerenje u njezina administratora. Ako administrator upravlja podacima i kodom, lokalna transakcija sama ne raspodjeljuje njegove ovlasti. Utrka je sada riješena unutar baze, ali A, B i C nisu u bazi: zahtjeve šalju preko mreže.

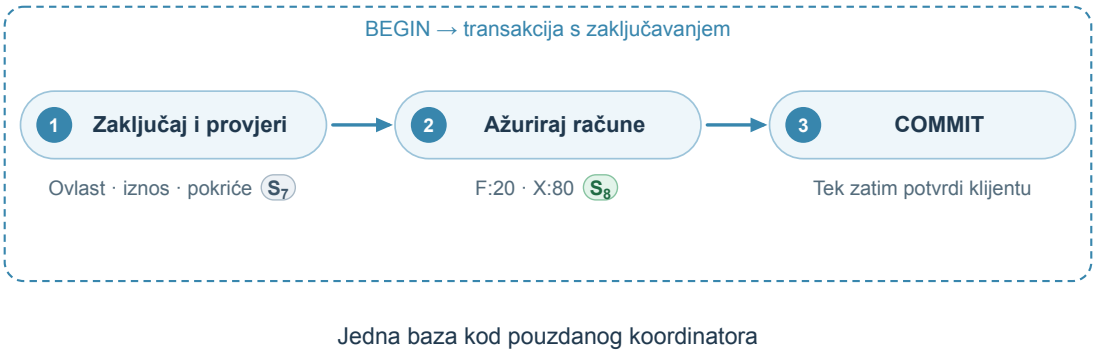


Figure 11: Jedna baza kod pouzdanog koordinatora

Nastavni rad (31–34 min): Neka studenti objasne dva odvojena kvara: pad između terećenja i odobrenja te dva zahtjeva koja provjere isti saldo. Povežite prvi s atomarnošću, drugi s izolacijom.

16 Raspodijeljenost: rad preko više procesa

I s jednim koordinatorom A, B i C šalju zahtjeve porukama preko mreže, a koordinator i baza mogu biti zasebni procesi. Takav sustav ima ime. Svaku komponentu koja se samostalno izvršava i razmjenjuje poruke zovemo čvor (*node*).

Definicija 1.1

Raspodijeljeni sustav (*distributed system*) čine komponente koje se neovisno izvršavaju te komuniciraju i koordiniraju razmjenom poruka radi zajedničkog cilja.

Ovaj odjeljak ne dodaje novi korak, nego imenuje ono što sustav već jest. Raspodijeljenost u ovom kolegiju promatramo kroz rad procesa koji razmjenjuju poruke, imaju lokalne informacije i mogu doživjeti djelomične ispade. Više procesa na istom računalu može imati neke od tih problema, ali procesi na više računala dodaju zasebne mrežne i strojne ispade. Broj fizičkih lokacija sam nije potpuna specifikacija sustava.

Na crtežu jasno razdvajamo klijenta, koordinatora i bazu. Čak je i aplikacija s jednom bazom i udaljenim klijentima raspodijeljena u komunikacijskom smislu. Kad kasnije dodamo replike baze, povećavamo raspodijeljenost pohrane; ne „pretvaramo" potpuno lokalni sustav u raspodijeljeni samo tim potezom. Kontrola i dalje može pripadati jednoj organizaciji. Ostaje pitanje što se događa ako koordinator kod A padne ili ga preopterete čitanja.

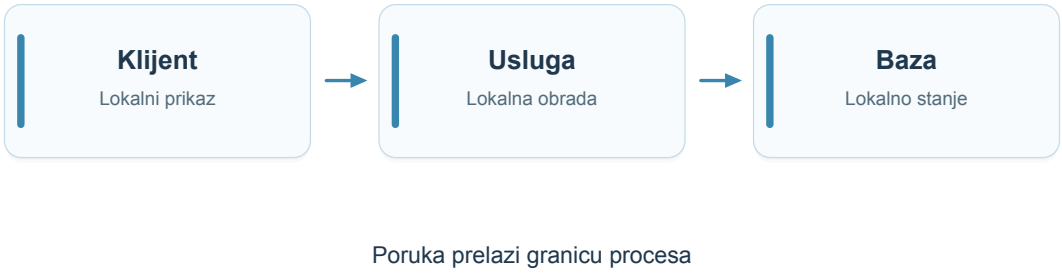


Figure 12: Poruka prelazi granicu procesa

Nastavni rad (34–36 min): Pitajte je li web aplikacija s jednom bazom i udaljenim korisnicima raspodijeljena. Obrazložiti granicu promatranja.

17 Zašto dodajemo replike?

Nastavljamo drugi korak, skaliranje koordinatora, u drugom obliku: dodatnim kopijama. Dosad sve isplate i sva čitanja prolaze kroz jednu bazu. Kako broj korisnika raste, prikazi salda i izvještaji mogu opteretiti primarnu bazu. Ako ona postane nedostupna, nemamo drugo mjesto spremno preuzeti njezin rad. To su razlozi za razmatranje replika: raspodjela upita za čitanje i dodatna ažurna kopija koja može pomoći oporavku.

Replika (*replica*) je kopija podataka koju sustav održava prijenosom promjena. Čvorove označavamo A, B i C prema organizaciji kod koje stoje, ali sve ih vodi isti operator; organizacije zasad nemaju vlastite ovlasti nad čvorovima. A zasad jedini prihvaća upise, a B i C primjenjuju prihvaćene promjene i mogu posluživati dio čitanja. Ne tvrdimo da time povećavamo broj neovisnih upisa. Ako A padne, dostupna replika može i dalje posluživati

dopuštena čitanja, ali nastavak upisa traži izbor nove primarne baze i zaštitu od istodobnog rada stare i nove primarne baze. Automatsko preuzimanje nije posljedica samog postojanja kopija.

U primjeru pretpostavljamo asinkrono slanje promjena nakon potvrde lokalnog upisa na A. Zato replika može kasniti; ako A nepovratno izgubi zapis prije slanja, preuzimanje sa zaostale replike može izgubiti potvrđenu promjenu. Politiku potvrđivanja i oporavka treba posebno odabrati. PostgreSQL 18: standby poslužitelji.

Sve kopije zasad vodi isti pouzdani operator. Poboljšavamo mogućnosti čitanja i oporavka, a ne raspodjelu upravljačkih ovlasti. Replika također nije zamjena za sigurnosnu kopiju: pogrešan upis ili brisanje može se prenijeti na sve replike. Zajednički kvar napajanja ili vjerodajnica može pogoditi više kopija. Sljedeći slajd zato pokazuje novu cijenu replikacije: različitu svježinu podataka.

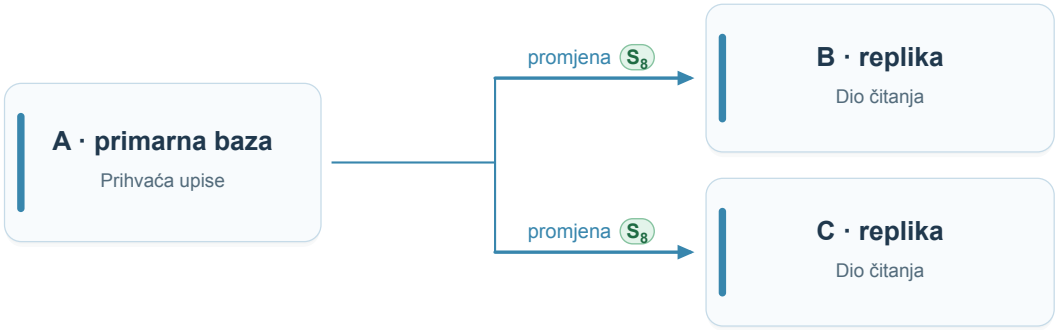


Figure 13: Više kopija za čitanje i oporavak; isti operator

Nastavni rad (36–39 min): Pitajte što dobivamo za čitanje, što za oporavak, a što za povjerenje u operatora. Jesu li dvije dodatne kopije dovoljne da upisi nastave nakon pada A?

18 Kopije nisu trenutačno jednake

Tablica prikazuje saldo fonda F i oznaku stanja na svakoj replici. U t_1 A prihvaća tx_1: fond ima 20, a X dobiva 80. B i C još prikazuju starih 100. U t_2 promjena stiže do B, a u t_3 do C. Odredišni račun X ažurira se kao dio iste promjene iako zbog preglednosti nije prikazan u tablici.

To je kašnjenje širenja jedne odluke, a ne sukob dvaju donositelja odluke. Kada se sve prihvaćene promjene dostave, kopije se u ovom izvođenju izjednače. Četiri retka nisu dokaz da će svaka buduća mrežna situacija završiti tako: potrebne su pretpostavke o dostavi i oporavku. Replika koja prikazuje 100 nema time pravo neovisno odobriti novu isplatu iz tih sredstava.

Trenutak	Replika A	Replika B	Replika C
t_0	100, S_7	100, S_7	100, S_7
t_1	20, S_8	100, S_7	100, S_7
t_2	20, S_8	20, S_8	100, S_7
t_3	20, S_8	20, S_8	20, S_8

Nastavni rad (39–41 min): Za svaki redak pitajte koliko korisnik vidi na C i koliko je prihvaćeno preostalo u fondu.

19 Korisnik ne vidi vlastitu isplatu

Gledamo iz perspektive korisnika organizacije B koji je upravo dobio potvrdu da je isplata 80 (tx_1) prihvaćena na A. Nakon uspjeha očekuje da sljedeći prikaz sadržava njegovu promjenu. Čitanje sa zaostale replike C može prikazati 100 umjesto 20. Jedno rješenje je čitati od autoriteta za upis, drugo tražiti da replika dosegne barem stanje S_8. Oba pristupa imaju trošak: čekanje, usmjeravanje ili privremenu nedostupnost čitanja.

Informativni izvještaj može dopustiti označeno kašnjenje. Konačna odluka o novoj isplati ne smije se osloniti samo na zastarjeli prikaz salda. Zato razdvajamo prikaz podataka i ovlast odlučivanja. Sučelje treba prikazati prihvaćenu isplatu i svježinu podataka bez stvaranja dojma da je stari saldo ponovno raspoloživ.



Figure 14: Za korisnika B staro čitanje izgleda kao poništena isplata

Nastavni rad (41–43 min): Tražite jedno čitanje gdje je kašnjenje prihvatljivo i jedno gdje bi bilo opasno.

20 Zahtjev je izvršen, potvrda nije stigla

Poruke ne kasne samo između kopija; mogu se i izgubiti. U ovom izvođenju zahtjev stigne do koordinatora. Koordinator provede tx_1: F ima 20, X ima 80, stanje je S_8 i šalje uspješan odgovor. Odgovor se izgubi ili toliko kasni da klijent prestane čekati. Klijent ne smije zaključiti da upisa nije bilo. Iz njegove perspektive isti timeout moguć je i kada zahtjev uopće nije stigao.

Ta neizvjesnost nastaje na granici sustava. Waldo et al. u odjeljku 4.3 opisuju djelomične ispade kao problem koji udaljeni poziv razlikuje od običnog lokalnog poziva: dio sustava može nastaviti raditi dok drugi dio ne odgovara. Za našu aplikaciju posljedica je konkretna: sučelje treba stanje „ishod nepoznat”, a ne izmišljenu konačnu poruku. A Note on Distributed Computing, odjeljak 4.3.

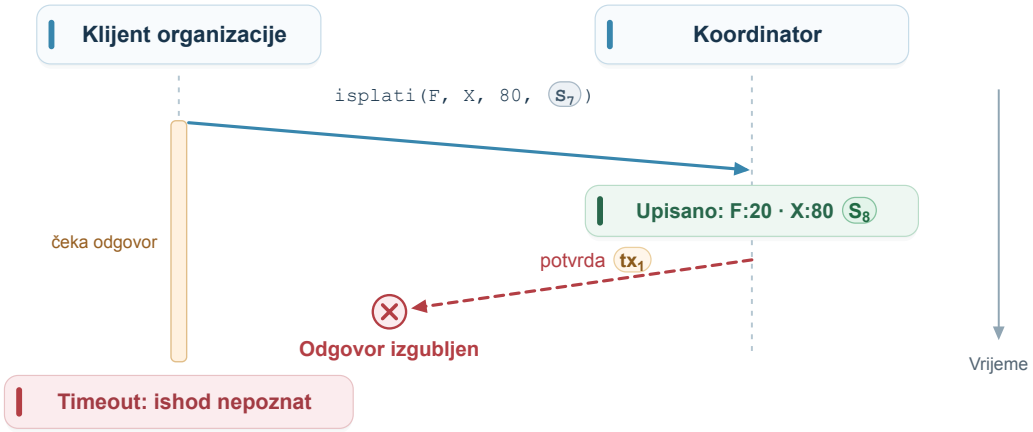


Figure 15: Timeout: isplata je upisana, a klijent ishod ne zna

Nastavni rad (43–46 min): Usporedite dva izvođenja s istim zaslonom klijenta. Pitajte koja činjenica nedostaje.

21 Ponovljeni pokušaj treba isti identitet

Klijent ne zna je li isplata prošla pa ponavlja zahtjev; bez zaštite bi ista isplata prošla dvaput. Zahtjevu zato dodajemo identifikator K42. Koordinator uz prihvaćenu isplatu bilježi rezultat vezan uz klijenta i taj identifikator. Ako isti klijent ponovi isti zahtjev, može dobiti raniji rezultat umjesto još jedne promjene. Takav je zahtjev idempotentan (*idempotent*): njegovo ponavljanje ne mijenja ishod. ID pritom ne smije biti ponovno iskorišten za drukčiji sadržaj: takav slučaj treba odbiti ili posebno obraditi.

Pohrana rezultata i promjena stanja moraju biti povezane tako da pad između njih ne stvori novu dvosmislenost. U nastavnom modelu pretpostavljamo atomarnu pohranu obaju zapisa i čuvanje ID-a tijekom promatranog razdoblja. Time ne tvrdimo da je mreža dostavila

poruku točno jednom. Ograničili smo učinak ponavljanja na strani usluge. AWS Builders' Library opisuje taj obrazac.

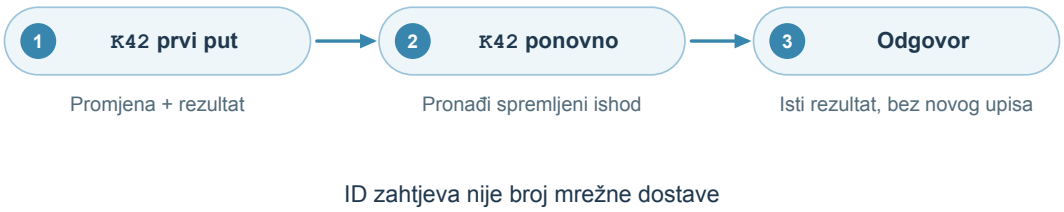


Figure 16: ID zahtjeva nije broj mrežne dostave

Nastavni rad (46–49 min): Pitajte što ako klijent za ponovni pokušaj stvori novi ID. Razlikujte ponovni pokušaj od nove korisničke namjere.

22 Što znači djelomičan ispad?

Djelomičan ispad (*partial failure*) znači da dio čvorova ili veza ne radi, a ostatak radi dalje. Izgubljena potvrda bila je jedan takav slučaj: klijent i koordinator rade, a veza među njima ne. U primjeru rade sva tri čvora, ali je prekinuta veza A–B. Čvor A normalno poslužuje svoje korisnike, dok čvor B do njega ne može doći. Čvor C komunicira s oboma. A misli da sustav radi, B misli da je A pao, a C dobiva odgovore od oba, ali ne zna da je veza A–B prekinuta. Nijedan čvor ne vidi stanje cijelog sustava; svaki zna samo ono što mu je stiglo porukama. „Sustav radi" zato nije dovoljno precizan opis: treba navesti tko koju operaciju može izvršiti. Za razliku od jedne funkcije u našem procesu, udaljena usluga može nastaviti mijenjati stanje i nakon što smo prestali čekati odgovor.

Izvor s Harvarda koristimo za temeljnu razliku, ali ne prenosimo njegove povijesne tvrdnje o tadašnjim proizvodima kao današnja ograničenja. Naš konkretni primjer pokazuje zašto i API i korisničko sučelje trebaju izraziti nepoznat ishod. Poboljšanje samo jedne komponente ne uklanja svaku neizvjesnost na komunikacijskoj granici.

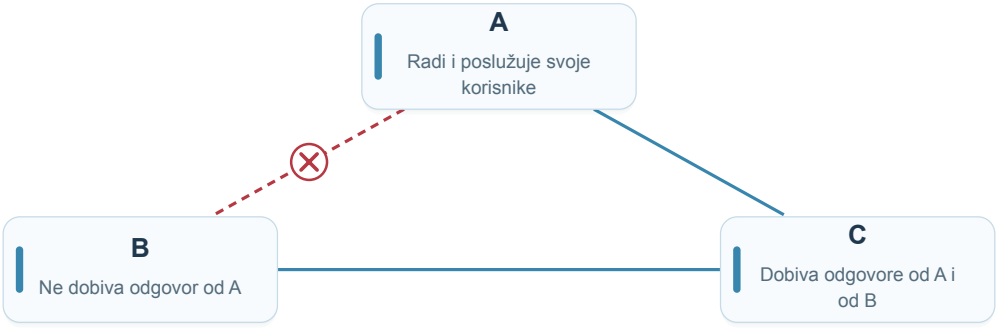


Figure 17: Radi li sustav? A kaže da, B kaže ne, a C ne zna za prekid A–B.

Nastavni rad (49–51 min): Pročitajte status iz perspektive A, zatim B. Isti raspored može izgledati drukčije različitim korisnicima.

23 Uz koji model tvrdimo da pravilo vrijedi?

Jamstvo koje želimo: **saldo fonda nikad ne postaje negativan ni kad poruke kasne ili čvor stane.** Prije nego što to tvrdimo, navodimo uvjete pod kojima to tvrdimo.

Model sustava navodi što dopuštamo okolini i kvarovima. Bez njega je tvrdnja „radi i kad nešto padne" neprovjerljiva. Za sada imamo tri poznata sudionika, ne proizvoljan broj anonimnih identiteta. Za promatrano izvršavanje nemamo poznatu gornju granicu dostave, a pošteni čvorovi slijede pravila dok rade. Zaustavljen čvor ne odgovara; ne šalje namjerno lažan podatak.

To ne znači da stvarne mreže uvijek kasne neograničeno. Znači da naš zaključak ne smije ovisiti o proizvoljno odabranom roku kao dokazu pada. Ako kasnije dodamo pretpostavku o oporavku ili o poznatoj granici, moramo je izričito navesti. Otpornost na ispad pohrane također nije dana besplatno; u ovom primjeru je posebno pretpostavljena.

Pretpostavka	**Za naš sljedeći primjer**
Članovi	A, B i C, unaprijed poznati
Poruke	mogu kasniti, biti ponovljene ili izgubljene
Vrijeme	nemamo poznat konačan rok dostave
Čvorovi	zasad mogu stati; ne izmišljaju poruke
Pohrana	trajnost lokalnog prihvaćenog zapisa pretpostavljena

Nastavni rad (51–54 min): Neka studenti dodaju jedan rizik koji model zasad ne pokriva. Očekivano: zlonamjeran administrator ili gubitak trajne pohrane.

24 Spor čvor i zaustavljen čvor mogu izgledati isto

Model ne daje poznat rok dostave, pa izostanak odgovora treba tumačiti oprezno. Pretpostavimo da B nije dobio odgovor od A tri sekunde. Jedno moguće izvođenje jest da je A ugašen. Drugo je da A radi, ali odgovor čeka u mreži. Treće je da je A završio posao i stao prije slanja odgovora. Iz samog izostanka poruke B ne zna koje se izvođenje dogodilo.

Timeout i dalje ima praktičnu vrijednost: omogućuje prestanak čekanja i pokretanje oporavka. No odluka da drugi čvor preuzme upise mora uzeti u obzir mogućnost da prvi i dalje radi. Kad bismo obojici dali neograničeno pravo konačnog potvrđivanja, mogli bismo dobiti dvije povijesti. Ovdje prepoznajemo problem; ne uvodimo još protokol izbora vođe.

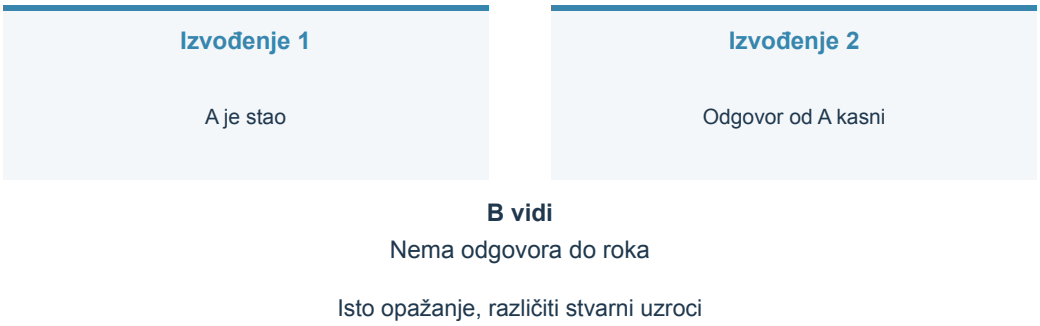


Figure 18: Isto opažanje, različiti stvarni uzroci

Nastavni rad (54–57 min): Tražite isti opaženi trag za spori i ugašeni čvor. Ne tvrditi da se kvar nikada ne može otkriti uz dodatne pretpostavke.

25 Mreža se razdvojila

Mrežna particija (*network partition*) u primjeru znači da poruke između skupine A i B te C ne prolaze. To ne zahtijeva gašenje svih računala. Obje strane mogu primati lokalne zahtjeve i imati podatke koji su bili jednaki neposredno prije razdvajanja. Ako nastave odlučivati svaka za sebe, više nema zajedničkog mjesta koje odmah vidi oba zahtjeva.

Važno je nacrtati konkretne nedostupne veze. Rečenica „internet je pao” skriva tko još s kim razgovara. Naš crtež koristi dva dijela radi jednostavnosti; stvarni rasporedi mogu biti složeniji. Za daljnji zadatak obje strane polaze iz F:100, S_7 i dobivaju zahtjeve za isplate 80 i 60.

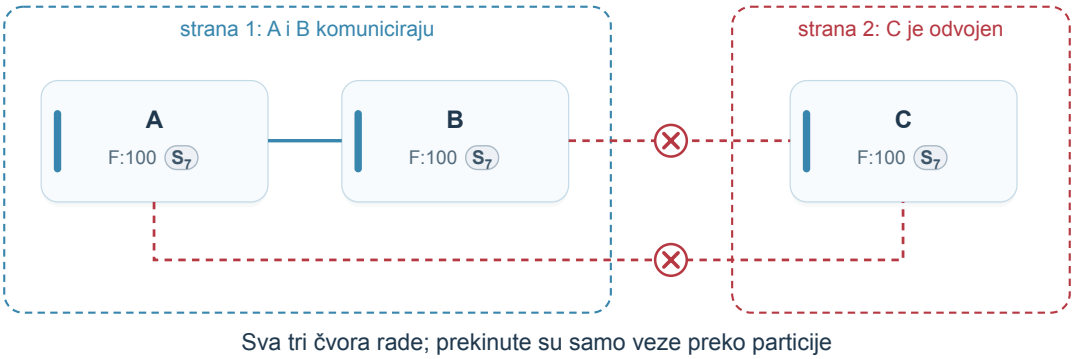


Figure 19: Sva tri čvora rade; prekinute su samo veze preko particije

Nastavni rad (57–60 min): Na ploči precrtajte veze prema C, ali ostavite korisnika kod C. Pitajte može li C još prikazivati lokalne podatke.

26 Dvije strane ne smiju obećati suprotno

Primarna baza i dalje je A, ali C više ne čuje A. Kao na slajdu o sporom i zaustavljenom čvoru, C ne može razlikovati pad od prekida veze: zaključi da je A pao i sam počne prihvaćati upise. Prije razdvajanja obje strane imaju F:100, X:0, Y:0 u S_7. A i B konačno potvrde tx_1 pa vide F:20, X:80, Y:0. C konačno potvrdi tx_2 pa vidi F:40, X:0, Y:60. Kada se veza vrati, ne možemo obje isplate uključiti u valjanu povijest iz početnih 100 bez novih sredstava. Njihovim zbrajanjem F bi pao na -40; odbacivanjem jedne morali bismo povući već izdanu konačnu potvrdu.

Odvojena strana može primati zahtjeve kao nepotvrđene prijedloge ili privremeno zaustaviti isplate. Drukčiji dizajn može unaprijed podijeliti prava potrošnje između strana, ali tada svaka više ne raspolaže cijelih 100. To nije pretpostavka našeg primjera. Pokazujemo zašto neovisne odluke nad istim raspoloživim sredstvima krše odabrano pravilo.

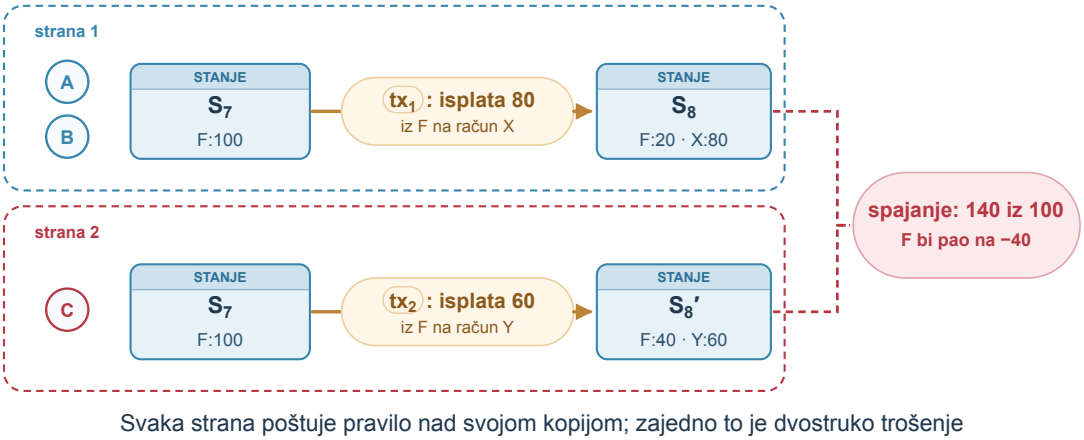


Figure 20: Svaka strana poštuje pravilo nad svojom kopijom; zajedno to je dvostruko trošenje

Nastavni rad (60–63 min): Rasprava: što smije pisati na gumbu i u potvrdi dok je zahtjev samo prijedlog?

27 Tri različita pitanja o arhitekturi

Dosad je sve vodio jedan operator, a organizacije ne žele da itko sam odlučuje o fondu. Prije nego ga uklonimo, imenujmo što točno uklanjamo. Raspodijeljenost opisuje razmještaj procesa, stanja i komunikacije. Decentralizaciju ovdje promatramo kao raspodjelu relevantnih ovlasti između neovisnih sudionika. P2P opisuje uloge u razmjeni usluga: sudionici nisu samo potrošači nego i pružatelji resursa. To su radne razlike za analizu, ne tvrdnja da postoji jedna univerzalna brojčana mjera decentralizacije.

RFC 5694 upozorava da P2P i centralizirani elementi mogu postojati u istom sustavu. Zato iz mreže vršnjaka ne zaključujemo automatski da nitko nema središnju kontrolu. Može postojati središnji upis članova ili jedini ključ za nadogradnju. Za svaki sustav pitamo koja funkcija ima koju strukturu. RFC 5694, odjeljci 2–4.

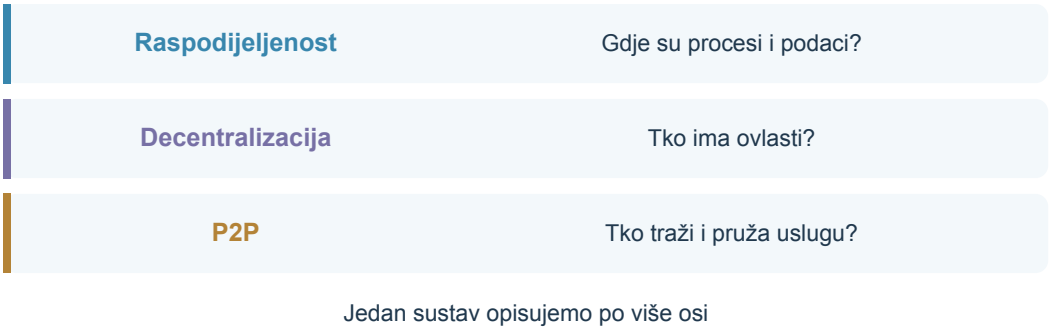


Figure 21: Jedan sustav opisujemo po više osi

Nastavni rad (63–66 min): Neka studenti za svaku riječ postave pitanje, umjesto da pokušaju složiti ljestvicu „više je bolje”.

28 Sto poslužitelja, jedan administrator

Ako jedna organizacija upravlja sa sto replika, raspodijelila je izvršavanje i možda povećala otpornost na pojedine kvarove. Time nije nužno raspodijelila ovlast promjene pravila. Jedan administratorski račun, zajednička nadogradnja ili odluka operatora može utjecati na sve kopije.

Za zajednički budžet zajednički operator može biti razumno ugovorno rješenje. Ako je problem samo dostupnost, dodatne replike mogu pomoći. Ako je problem da operator smije neprimjetno prepisati povijest, sto njegovih replika ne uklanja taj izvor povjerenja. Pitanje neovisnosti odnosi se na kontrolu relevantnu za prijetnju koju analiziramo, a ne samo na broj kutija na dijagramu.

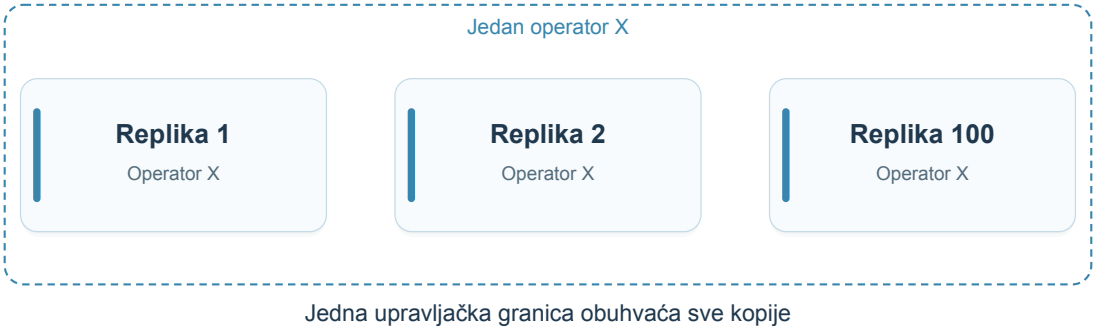


Figure 22: Jedna upravljačka granica obuhvaća sve kopije

Nastavni rad (66–69 min): Prekrijte sve čvorove jednom granicom „jedan administrator”. Pitajte što ostaje neriješeno.

29 P2P opisuje razmjenu, ne jamči dogovor

P2P (*peer-to-peer*) označava ravnopravne čvorove koji i traže i pružaju uslugu. U P2P dijelu aplikacije A može tražiti zapis od B, a zatim drugom sudioniku pružiti podatak koji ima. To razlikujemo od stroge podjele na klijente koji samo traže i poslužitelje koji samo pružaju uslugu. Ne zahtijevamo da svaki čvor bude povezan sa svakim drugim niti da svi imaju jednake resurse.

Ni P2P razmjena ni topologija trokuta ne definiraju pravilo prihvaćanja. A i B mogu vrlo učinkovito razmijeniti dvije suprotne tvrdnje. Treba dodatno odrediti članstvo, ovlasti, provjeru i dogovor. Primjer je nastavna primjena razlike iz RFC-a, a ne tvrdnja o određenom današnjem proizvodu.

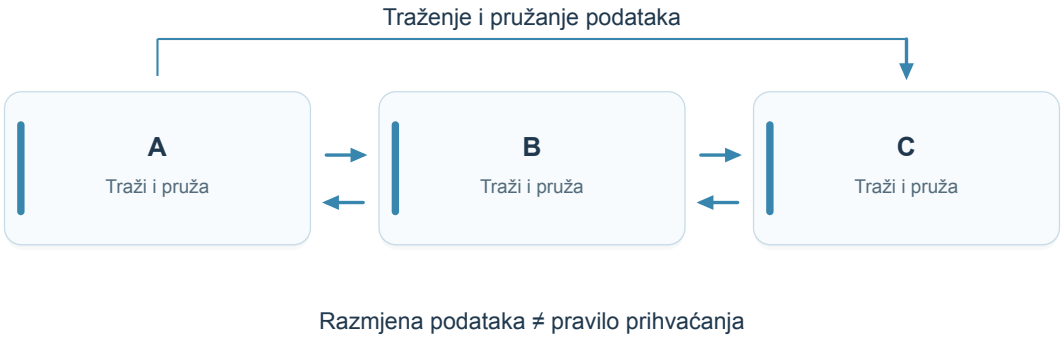


Figure 23: Razmjena podataka ≠ pravilo prihvaćanja

Nastavni rad (69–71 min): Nacrtajte strelice u oba smjera i zatražite da student navede informaciju koja još nedostaje.

30 Raspodijeljen, decentraliziran ili P2P?

Zadatak traži obrazloženje, ne samo naljepnicu. Prvi sustav ima raspodijeljenu pohranu, ali opisana kontrola ostaje kod tvrtke. Drugi raspodjeljuje odobravanje između organizacija, a poslužitelj, baza i kod, pa time izvršavanje i dostupnost, ostaju kod jedne tvrtke. Treći ima P2P razmjenu datoteka i središnju ovlast za članstvo.

Za neke osi tekst nema dovoljno podataka. Primjerice, iz činjenice da tri organizacije odobravaju promjene ne znamo jesu li te ovlasti tehnički provedive ili samo ugovorne. Dobar odgovor navodi što znamo, što ne znamo i koje bismo dodatno pitanje postavili. Time vježbamo disciplinu važniju od prepoznavanja buzzworda.

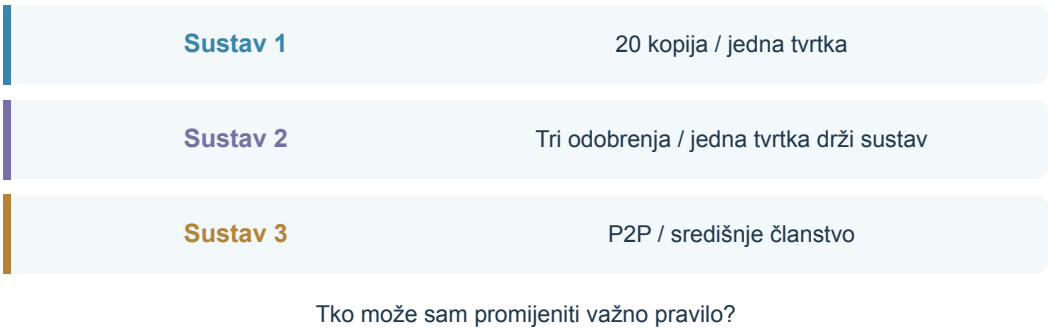


Figure 24: Tko može sam promijeniti važno pravilo?

Nastavni rad (71–74 min): Dvije minute u parovima, zatim po jedan zaključak za svaki sustav. Priznati „nedovoljno informacija" uz dobro dodatno pitanje.

31 Nova pretpostavka: nema zajedničkog operatora

Definicija 1.2

Decentralizirani sustav (*decentralized system*) je raspodijeljeni sustav u kojem su ovlasti nad zajedničkim stanjem (upis, potvrda, pravila, članstvo) podijeljene između neovisnih sudionika, tako da nijedan sam ne može nametnuti promjenu.

Svaki decentralizirani sustav je raspodijeljen, ali obrnuto ne vrijedi: sto replika pod jednim administratorom raspodijeljeno je, a nije decentralizirano. Ovu definiciju zasnivamo na ovlastima; Buterin (2017) to zove političkom decentralizacijom, za razliku od arhitekturne (broj računala). Van Steen i Tanenbaum (2023, §1.1) razliku opisuju drukčije, kao nužnu nasuprot dovoljnoj raspodjeli procesa i resursa.

Sada mijenjamo zahtjev, a ne samo tehnički raspored. Organizacije ne žele dati jednoj organizaciji ovlast konačnog odlučivanja. Svaki drži svoju kopiju i provodi dogovorena pravila. Neovisnost upravljanja donosi novu zadaću: treba se usuglasiti ne samo oko sadržaja poruke nego oko toga koja je promjena prihvaćena.

Mreža i dalje može kasniti, a čvorovi i dalje mogu stati. Uz to, različiti operatori mogu imati različite interese. To ne znači da svi moraju biti zlonamjerni. Model mora objasniti što se događa ako se netko ne pridržava protokola. Decentralizacija ovdje premješta dio povjerenja iz jedne organizacije u pravila i pretpostavke zajedničkog postupka.



Figure 25: Dogovor mora povezati neovisne operatore

Nastavni rad (74–77 min): Pitajte koja se pretpostavka promijenila u odnosu na slajd sa sto poslužitelja. Nije se samo povećao broj strojeva.

32 Povjerenje rastavljamo na konkretne ovlasti

Koordinatoru su sve organizacije vjerovale u svemu: upisu, redoslijedu, kodu i oporavku. Bez zajedničkog operatora povjerenje ne nestaje, nego se dijeli: za svaku ovlast treba reći kome je povjerena. „Ne vjerujemo A-u" preširoka je izjava. Možemo vjerovati A-u da ispravno identificira svoje zaposlenike, ali mu ne dati pravo da sam briše tuđe zapise. Možemo dopustiti operatoru slanje poruka, ali od njega ne preuzimati konačnu odluku bez provjere. Treba opisati ovlasti i posljedice njihove zlouporabe.

Pravila su ovdje dio programa koji provjerava svaki zahtjev, npr. da isplata ima pokriće i ne prelazi limit; tko smije promijeniti taj kod, može promijeniti i pravilo. Članstvo je popis organizacija koje smiju podnositi i potvrđivati zahtjeve (A, B, C), a ne broj poslužitelja: prijem nove organizacije D ili isključenje C mijenja tko odlučuje.

Za budžet sastavljamo malu matricu: tko podnosi zahtjev, tko provjerava lokalne uvjete i tko potvrđuje prihvaćanje. Zasebno razmatramo promjenu popisa članova i nadogradnju koda. Ako ta dva koraka ostanu pod jednim ključem, ondje je važna središnja ovlast bez obzira na način razmjene podataka.

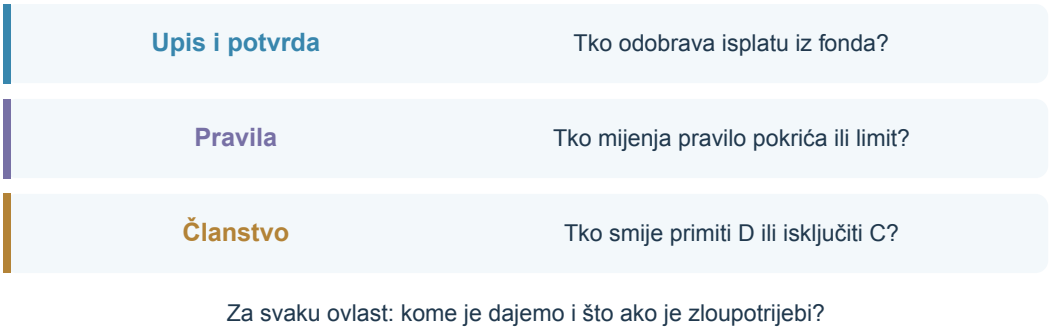


Figure 26: Za svaku ovlast: kome je dajemo i što ako je zloupotrijebi?

Nastavni rad (77–80 min): Neka studenti za svaku ovlast navedu jedan rizik i osobu/organizaciju kojoj bi je dali.

33 Isti pošiljatelj govori dvije stvari

Koordinatora više nema: svaka organizacija sama odobrava ili odbija prijedloge. Što ako A tu ovlast zloupotrijebi? B predlaže tx_1, C predlaže tx_2, a A odstupa od pravila i odobrava oba nespojiva prijedloga. B dobiva A-ovo odobrenje za tx_1: 80 za X, a C za tx_2: 60 za Y. Zasad pretpostavljamo da se poruka u tuđe ime ne može krivotvoriti, pa oba znaju da poruka stvarno dolazi od A. Ipak, obje isplate ne stanu u početni budžet od 100. Čak i kada A navede pojedinačno ispravan novi saldo za svaki zahtjev, ukupna obećanja nemaju pokriće.

Potrebna je dodatna razmjena informacija i pravilo zajedničkog prihvaćanja. Informacija „A je poslao” nije dovoljna. Kod zaostale replike postoji jedna prihvaćena odluka i različita svježina znanja. Ovdje postoje nespojive tvrdnje o tome što je konačno prihvaćeno.

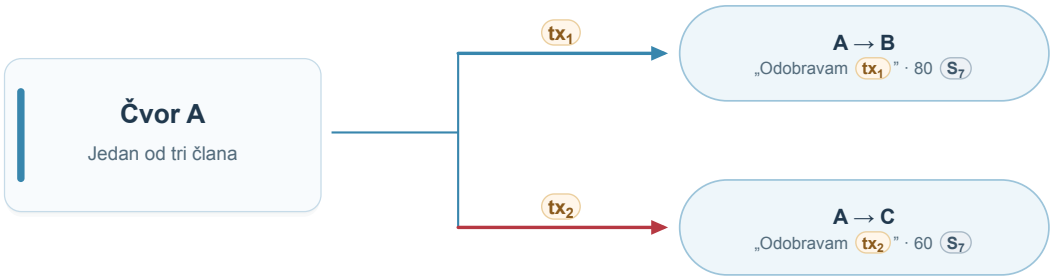


Figure 27: Poruka je stvarno od A, ali to ne čini odluku ispravnom

Nastavni rad (80–83 min): Na ploči razdvojite pogled B-a i C-a. Što svaki od njih može zaključiti bez dodatne poruke?

34 Pad i proizvoljno ponašanje nisu isti kvar

Ono što je A upravo napravio, slanje nespojivih potvrda različitim primateljima, zove se bizantsko ponašanje (*Byzantine behaviour*). U modelu pada čvor prestane izvršavati korake; ostatak sustava mora raditi s manjkom odgovora. Bizantski model dopušta proizvoljno odstupanje od protokola, uključujući selektivno slanje, lažne vrijednosti i međusobno proturječne tvrdnje. Uzrok može biti napad, programska pogreška ili kvar; za ponašanje protokola važna je dopuštena radnja, ne moralna ocjena operatora.

Lamport, Shostak i Pease uvode problem pouzdanog dogovora kad neispravne komponente različitim dijelovima sustava daju proturječne informacije. Ovdje uzimamo tu razliku, a njihove uvjete i algoritme ostavljamo za sljedeće predavanje. Ne zaključujemo prag otpornosti bez specifikacije komunikacije, identiteta i protokola. The Byzantine Generals Problem, uvod.

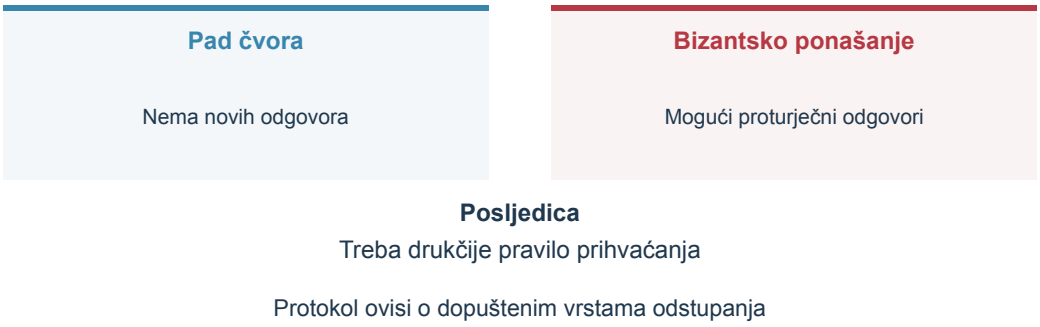


Figure 28: Protokol ovisi o dopuštenim vrstama odstupanja

Nastavni rad (83–86 min): Pitajte mora li bizantski kvar biti namjerna prijevara. Ne mora.

35 Kašnjenje ili proturječje?

U prvom slučaju opažanje je kompatibilno s poštenom, zaostalom replikom. Samo iz starog stanja ne zaključujemo da C laže. U drugom slučaju imamo konkretne proturječne potvrde koje krše pravilo, ako vjerodostojno znamo njihovo podrijetlo i značenje. Ne mora svaka razlika u zaslonima biti dokaz istog problema.

Važno je odrediti koji dokaz imamo. „Moj ekran pokazuje drugo” slabije je od dvije poruke koje potvrđuju dvije nespojive konačne isplate iz istog stanja. Čak ni detekcija proturječja

ne određuje automatski koja promjena vrijedi. To treba urediti protokol i način oporavka. Primjer priprema studente da pri analizi ispada ne zamijene opažanje za objašnjenje.

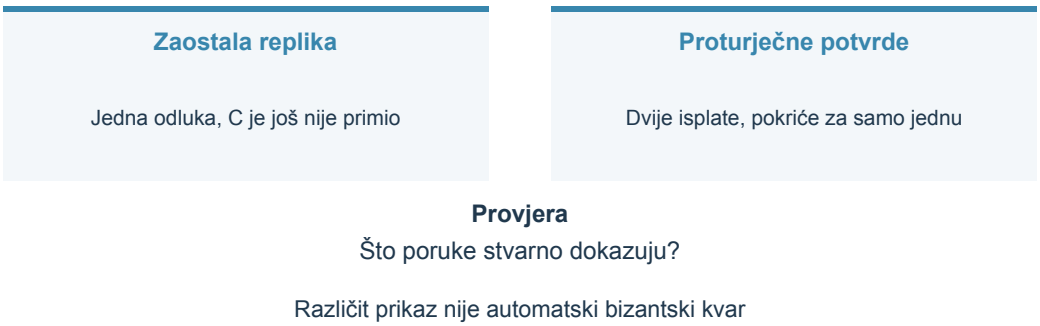


Figure 29: Različit prikaz nije automatski bizantski kvar

Nastavni rad (86–89 min): Neka studenti klasificiraju oba slučaja i navedu što bi još zatražili za dijagnozu.

36 Identitet ne zamjenjuje pravilo prihvaćanja

Dosad smo prešutno pretpostavljali da poruka koja kaže da je od A doista dolazi od A. Mreža to sama ne provjerava: poruku „odobravam tx_1" mogao bi poslati bilo tko i potpisati se kao A. Zasad pretpostavljamo da krivotvorenje pošiljatelja nije moguće; to omogućuje digitalni potpis, koji obrađujemo u predavanju 03. Ni to nije dovoljno. Provjeren identitet ne daje sadržaju automatsku valjanost. Poznati član može poslati zahtjev za isplatu iz stanja koje više nije aktualno, tražiti radnju za koju nije ovlašten ili dati suprotne odgovore različitim primateljima.

Provjera zato ima više slojeva: podrijetlo, dopuštenost radnje prema stanju i dogovor da je promjena prihvaćena. Ova razlika štiti od česte prečice da je svaka autentificirana poruka ujedno zajednička odluka. Mehanizme potpisa obradit ćemo kad budu potrebni za provjerljive zapise.

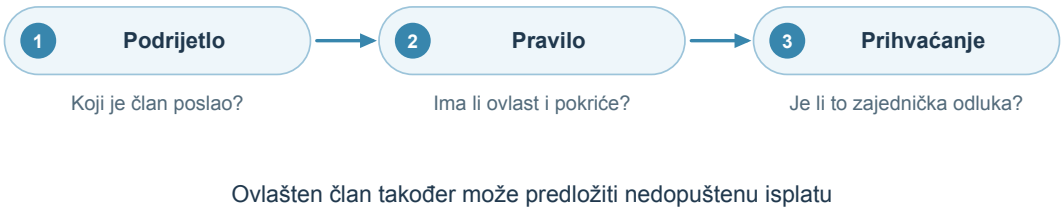


Figure 30: Ovlašten član također može predložiti nedopuštenu isplatu

Nastavni rad (89–91 min): Nemojte uvoditi hash funkcije ni sintaksu potpisa. Studenti trebaju samo prepoznati tri različita pitanja.

37 „Uzmimo dva od tri" još nije protokol

Ako potvrda jednog člana nije dovoljna, prirodan je pokušaj tražiti većinu: dva od tri. Neka B prihvati tx_1 kad vidi dvije potvrde, svoju i A-ovu. C prema istom pravilu prihvati tx_2 jer vidi svoju i A-ovu potvrdu. A je potvrdio oba zahtjeva. Obje skupine imaju po dva od tri člana, ali jedini član u njihovu presjeku odstupa od protokola. Zato pravilo „većina glasova" u ovom naivnom primjeru nije osiguralo jedinstven rezultat.

Ovo nije dokaz da svaki dogovor s tri sudionika uvijek mora propasti, niti izvod općeg praga tolerancije. To je protuprimjer točno navedenom postupku. Za jamstvo treba opisati tko i kada smije glasati, koje poruke razmjenjuje, kakve kvarove toleriramo i pod kojim uvjetima čekanje završava. Tu ćemo u predavanju 02 formalizirati problem konsenzusa.



Figure 31: Dva lokalna skupa glasova, dva različita zaključka

Nastavni rad (91–95 min): Nacrtajte skupove {A,B} i {A,C}. Tražite da student pokaže zašto sam presjek nije dovoljan.

38 Tko uopće ima pravo glasa?

„Dva od tri" pretpostavlja da znamo tko su ta tri. Naš primjer cijelo vrijeme ima dogovoreno članstvo. Ako primimo deset poruka od A, to nisu glasovi deset organizacija. Jednako tako deset procesa kojima upravlja A ne znači deset neovisnih članova. Pravilo glasovanja mora vezati glas uz određeni identitet i definirati promjene tog skupa.

Otvoreno pristupanje postavlja dodatno pitanje: kako spriječiti da jedna strana stvori proizvoljan broj identiteta i tako promijeni ishod? Danas to samo izdvajamo kao novo ograničenje. Ne miješamo model bizantskog ponašanja poznatih članova s otpornošću na stvaranje identiteta. Precizno članstvo jedna je od stavki s kojom završavamo specifikaciju za sljedeće predavanje.

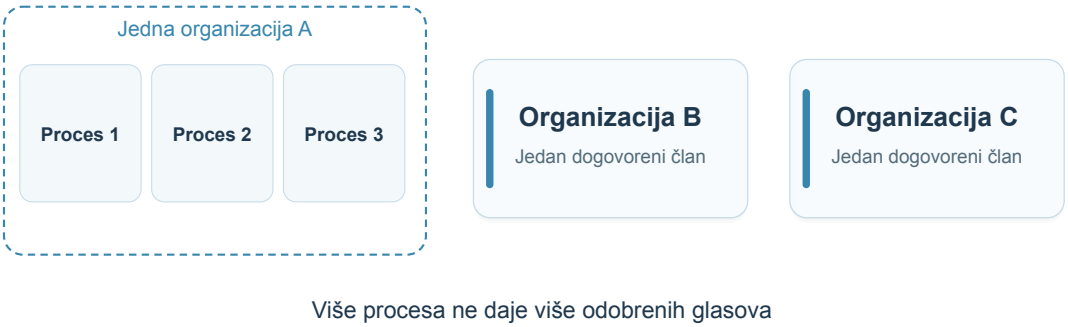


Figure 32: Više procesa ne daje više odobrenih glasova

Nastavni rad (95–97 min): Pitajte što je glas u našem modelu: paket, proces, račun ili organizacija? Odgovor ovisi o dogovorenom članstvu.

39 Studija slučaja: dvije isplate iz istog budžeta

Isti par isplata, tx_1: 80 → X i tx_2: 60 → Y, izvodimo tri puta. Svaki put pratimo samo poruke koje su stvarno stigle i lokalne zapise sudionika.

39.1 Pošten koordinator

Polazno stanje je F:100, X:0, Y:0 u S_7. B šalje tx_1, C šalje tx_2, a A je jedini ovlašteni donositelj odluke. A prima tx_1 prvi, prihvaća ga i stvara S_8; tx_2 i dalje očekuje S_7, pa ga odbija (a pokrića ionako ne bi bilo). Obje strane dobiju izričit odgovor i svi završe s istim stanjem. Pravilo je očuvano jer postoji jedno mjesto koje vidi oba zahtjeva i određuje redoslijed. Cijena: svi ovise o A.

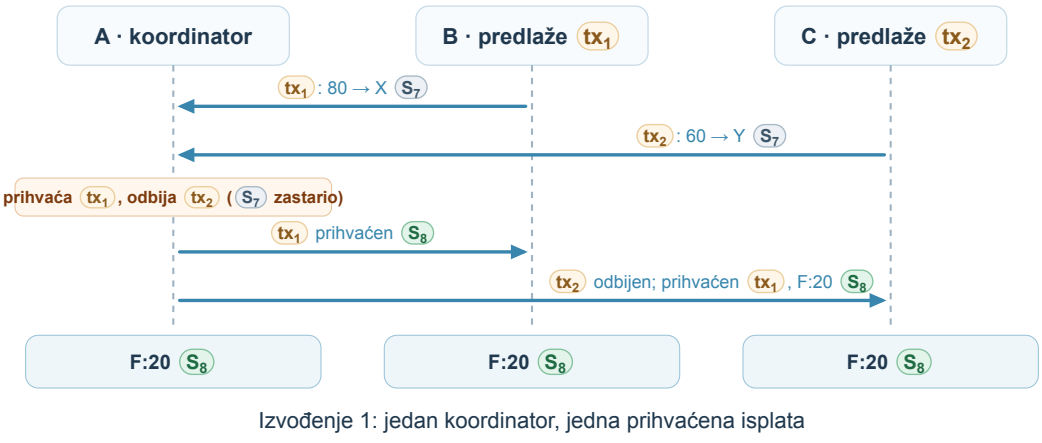


Figure 33: Izvođenje 1: jedan koordinator, jedna prihvaćena isplata

39.2 Izgubljena potvrda

Odluka je ista, ali poruka „tx_1 prihvaćen” prema B se gubi. Stanje kod A je ispravno: jedna isplata, F:20. B međutim i dalje prikazuje F:100 · S_7 i ne zna je li tx_1 prihvaćen, odbijen ili samo spor. Ako B ponovno pošalje tx_1, A ga mora prepoznati kao isti zahtjev, a ne kao novu isplatu. Ovdje nije prekršeno pravilo fonda, nego je nepotpuno znanje klijenta. To je drukčiji problem od dvostrukog trošenja i traži drukčiji popravak: ponovno slanje, identifikator zahtjeva i upit o stanju.

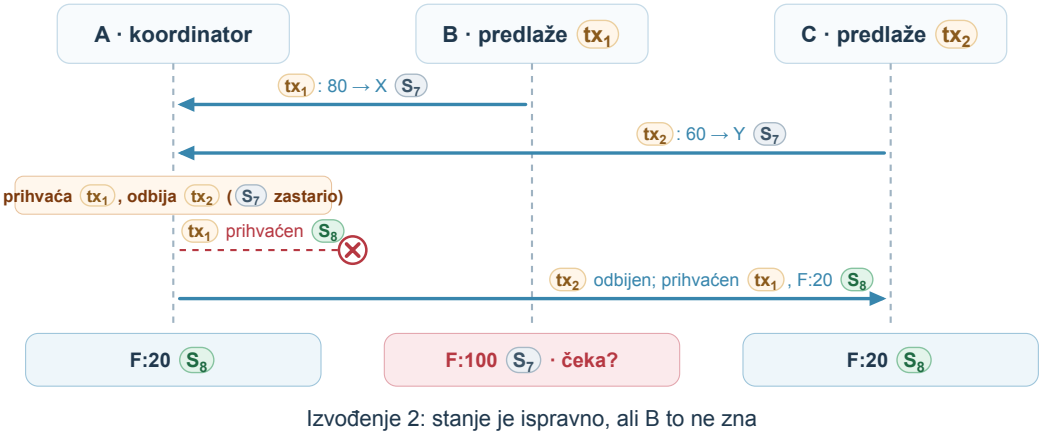


Figure 34: Izvođenje 2: stanje je ispravno, ali B to ne zna

39.3 Pravilo „dva od tri”

Sada nema koordinatora. Naivno pravilo kaže: isplata je konačna kad je podrže dva od tri sudionika. B glasa za svoj tx_1 i pita A; C glasa za svoj tx_2 i pita A. A odgovara DA obojici. B broji B + A i proglašava tx_1 konačnim; C broji C + A i proglašava tx_2 konačnim. Ukupno je isplaćeno 140 iz fonda od 100: dvostruko trošenje. Dva skupa od po dva glasa preklapaju se samo u A, a A je rekao dvije različite stvari. Pravilo „dva od tri” jamči sigurnost samo ako sudionik u presjeku ne smije glasati dvaput; to je tema predavanja 02.

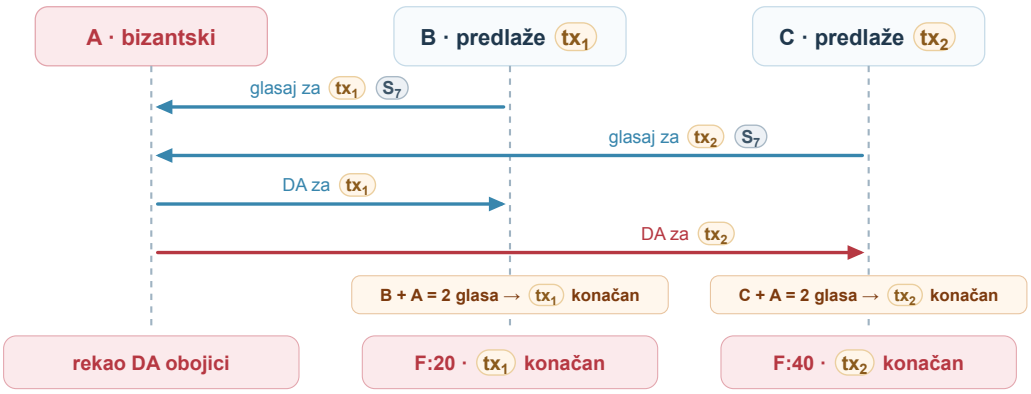


Figure 35: Izvođenje 3: pravilo „dva od tri" daje dvije konačne odluke, 140 iz 100

Nastavni rad (97–103 min): Nastavnik vodi tri izvođenja kroz ilustracije, po 2 minute. Prije otkrivanja ishoda pitajte što će svaki sudionik zapisati.

40 Analiza: gdje je pravilo o saldu prekršeno?

U prvom izvođenju postoji jedan ovlašteni donositelj odluke i jedna prihvaćena isplata. U drugom stanje može biti ispravno, a znanje klijenta nepotpuno. U trećem naivno pravilo dopušta dvije konačne odluke. Te razlike treba imenovati prije prijedloga popravka. „Dodaj još jednu kopiju" ne objašnjava kako će se promijeniti pravilo prihvaćanja.

Za svaki prijedlog popravka tražimo preostalu slabost. Ako svi čekaju sve, što radi zaustavljen član? Ako netko odlučuje sam, kome je povjereno upravljanje? Ako glasujemo, koje članstvo i dopuštene kvarove pretpostavljamo? Odgovori pokazuju zašto je dogovor o redoslijedu poseban problem, a ne dodatna strelica na arhitekturnom crtežu.

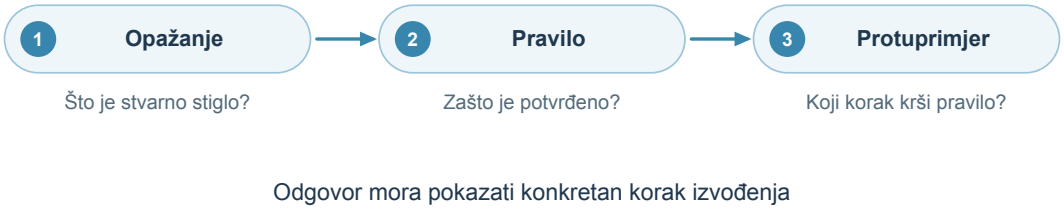


Figure 36: Odgovor mora pokazati konkretan korak izvođenja

Nastavni rad (103–107 min): Prođite tri izvođenja iz studije slučaja. Pohvaliti konkretan protuprimjer, ne složenost predloženog protokola.

41 Odaberite arhitekturu prema problemu

Uspoređujemo potrebe, ne dodjeljujemo moralne ocjene arhitekturama. Zajednička baza ima manje sudionika u odlučivanju i može biti dobar odgovor kad je operator prihvaćen. Replikacija rješava dio problema rada s kopijama i oporavka, ali traži pravilo potvrđivanja i preuzimanja. Neovisno odlučivanje više organizacija uvodi dodatnu razmjenu poruka, složenije kvarove i pitanje zajedničke nadogradnje.

Za budžet pitamo treba li svaka organizacija tehnički provjeravati odluke ili je dovoljan ugovor i revizija rada operatora. Ne postoji obveza koristiti *blockchain* samo zato što je tema kolegija. Danas još nemamo njegov mehanizam; imamo jasniji problem koji bi odabrana tehnologija trebala opravdano riješiti.

Potreba	**Početni smjer**
Svi prihvaćaju jednog operatora	zajednička služba i baza
Treba preživjeti pad pojedine kopije	replikacija i plan oporavka
Neovisni operatori moraju provjeravati odluke	zajednički protokol i model kvarova

Više koordinacije ima cijenu.

Nastavni rad (107–110 min): Za naš budžet tražite dva različita, ali obrazložena izbora. Dodatne pretpostavke zapišite uz izbor.

42 Ponavljanje: što ne štima u ovom prijedlogu protokola?

Vratimo se budžetu: fond F ima 100 jedinica, B predlaže tx_1 (80 iz F na račun X), a C predlaže tx_2 (60 iz F na račun Y). Svaka isplata zasebno ima pokriće, ali zajedno nemaju. Netko predlaže: „Isplatu spremamo na tri poslužitelja, pa je sustav decentraliziran. Čim jedan poslužitelj potvrdi, isplata je prihvaćena. Ako potvrda ne stigne na vrijeme, isplata nije prošla, pa šaljemo novi zahtjev.” Prijedlog zvuči uvjerljivo, ali sadrži tri neobrazložena zaključka. Svaki se ruši scenarijem koji smo već vidjeli.

Prvo, „decentraliziran”: broj kopija ne govori tko upravlja. Tri poslužitelja pod jednim administratorskim računom i dalje su jedan centar kontrole. Drugo, „prihvaćena”: tx_1 i tx_2 polaze od stanja S_7. Ako poslužitelj 1 prvi potvrdi tx_1, a poslužitelj 2 prvi potvrdi tx_2, i A i B misle da su uspjeli, a fond bi trebao isplatiti 140 od 100 jedinica. Treće, „nije prošla”: istek roka ne znači neuspjeh. Isplata je možda izvršena, a izgubila se samo potvrda; novi zahtjev s novim identitetom tada se izvršava kao druga isplata.

Popravak ima tri dijela: izričito pravilo kada je isplata prihvaćena, isti identitet pri ponovnom pokušaju i odvojena tvrdnja o tome tko upravlja poslužiteljima. Ni ta poboljšanja zajedno ne čine cjelovit protokol tolerantan na bizantske kvarove.

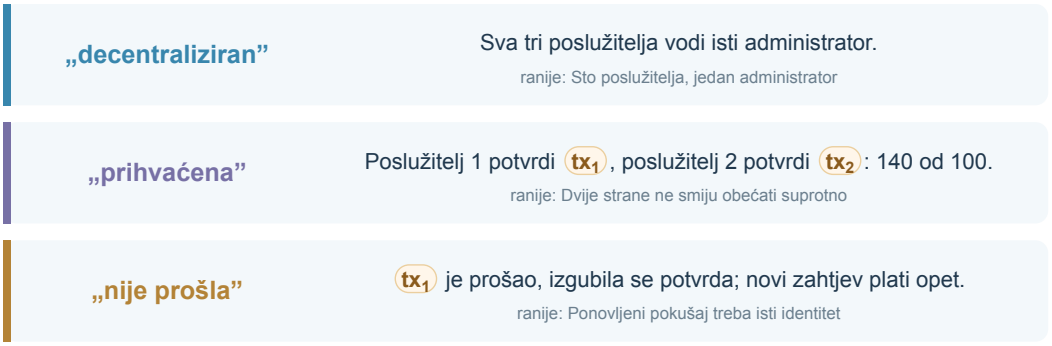


Figure 37: Popravak: pravilo prihvaćanja, isti ID pokušaja i izričito tko upravlja.

Nastavni rad (110–112 min): 2 min u parovima, zatim 2 min rasprave uz protuprimjere.

43 Sažetak: tri pojma koja ne smijemo izjednačiti

Predavanje je krenulo od jednog zapisa i poslovnog pravila. Lokalna atomarna promjena riješila je sukob u jednoj bazi. Replike su otvorile pitanje svježine, a mreža neizvjesnost o ishodu. Neovisni operatori promijenili su granicu povjerenja i dodali mogućnost proturječnih poruka. Svaki korak zahtijevao je novu, izričitu pretpostavku.

Provjerite možete li objasniti razliku između raspodijeljenosti, decentralizacije i P2P-a bez navođenja imena proizvoda. Zatim objasnite zašto su zaostala kopija i dvije suprotne konačne potvrde različiti problemi. Konačno, navedite što u našem primjeru znamo o članovima i vremenu dostave. To je podloga na koju će se nasloniti precizni zahtjevi konsenzusa.

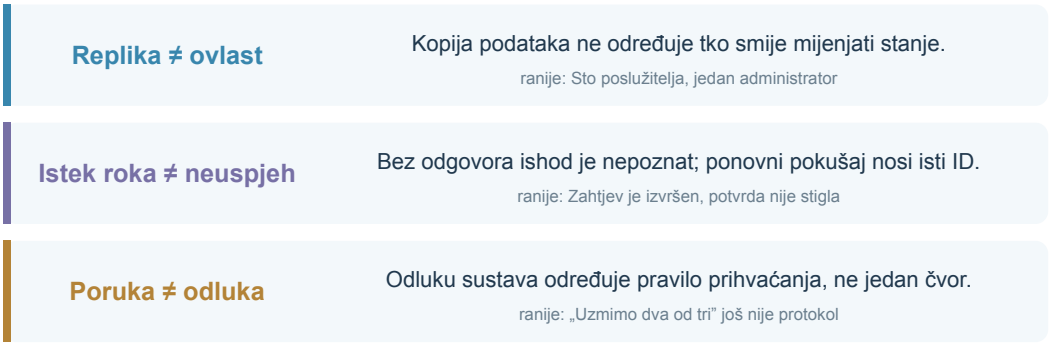


Figure 38: Jamstvo vrijedi samo uz izričit model: ovlasti, kvarovi i mreža.

Nastavni rad (114–116 min): Brzi izlazni odgovori: jedno pitanje po redu učionice, bez ponovnog čitanja definicija.

44 Otvoreno pitanje za predavanje 02

Prva polovica predavanja pretpostavljala je jednog operatora: koordinator, baza i replike bili su pod istom upravom. Od odjeljka „Nova pretpostavka: nema zajedničkog operatora" organizacije A, B i C vode čvorove neovisno, a jedna od njih može slati proturječne poruke. Pokazali smo koja pravila u tom slučaju ne rade; pravilo koje radi još nemamo. To je pitanje s početka predavanja: banke se oslanjaju na središnju banku, a mi tražimo dogovor bez nje. Otvoreno pitanje glasi: kako ispravni sudionici prihvaćaju istu promjenu i kada smiju prestati čekati?

Za sljedeće predavanje predajemo specifikaciju, a ne gotov odgovor: postoje poznati sudionici, dopuštena odstupanja i zahtjev da ispravni sudionici ne donose nespojive konačne odluke. Treba još precizno odrediti koje su odluke valjane i pod kojim uvjetima postupak mora završiti. Ako zaboravimo bilo koji od tih dijelova, lako je predložiti prividno rješenje koje uvijek čeka ili uvijek prihvaća unaprijed zadanu vrijednost.

Za pripremu nacrtajte dva prijedloga: jedan koji čeka sve odgovore i jedan koji odlučuje nakon prvih nekoliko. Za svaki napišite primjer u kojem ima problem. Ne traži se naziv algoritma niti programska implementacija. Traži se veza između uvjeta rada i jamstva koje tvrdite da pružate.

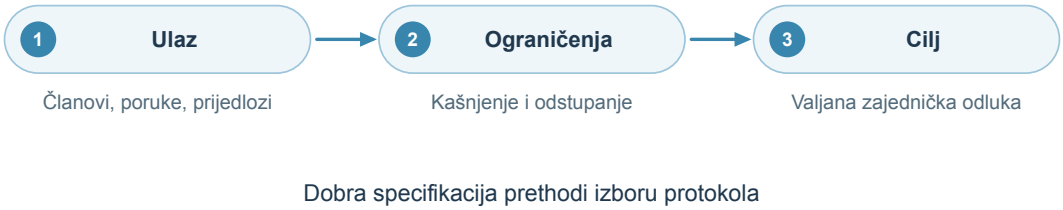


Figure 39: Dobra specifikacija prethodi izboru protokola

Nastavni rad (116–119 min): Ostavite minute za završna pitanja. Najavite konsenzus i bizantske generale u 02, bez preuzimanja sljedećeg predavanja.

45 Literatura za produbljivanje

Za prvo čitanje odaberite Waldoov odjeljak o djelomičnim ispadima i usporedite ga s primjerom izgubljene potvrde. Iz RFC-a izdvojite jednu funkciju sustava koja može biti središnja čak kad je razmjena P2P. U uvodu rada o bizantskim generalima prepoznajte problem različitih informacija poslanih različitim primateljima. Algoritamski dio tog rada nije preduvjet za današnju vježbu.

AWS-ov tekst povezuje ponavljanje zahtjeva s API dizajnom. Primjeri i dijagrami našeg digitalnog budžeta autorski su nastavni primjeri; ne prikazuju implementaciju AWS-a niti platnog sustava u produkciji. Precizna evidencija izvora, pregledanih dijelova i adaptacije starog BA PDF-a nalazi se u sources.md uz predavanje.

- Waldo et al.: A Note on Distributed Computing, §4.3.
- RFC 5694: P2P Architecture, §2–4.
- Lamport, Shostak, Pease: The Byzantine Generals Problem, uvod.
- AWS: Making retries safe with idempotent APIs.

Nastavni rad (119–120 min): Objasnite što čitati iz kojeg izvora. Literatura nije dodatna lista obveznih proizvoda.